

Caseware Take-Home — Architecture & Design

Jean Paul Forero · 16 September 2026

0. The constraint that shapes everything

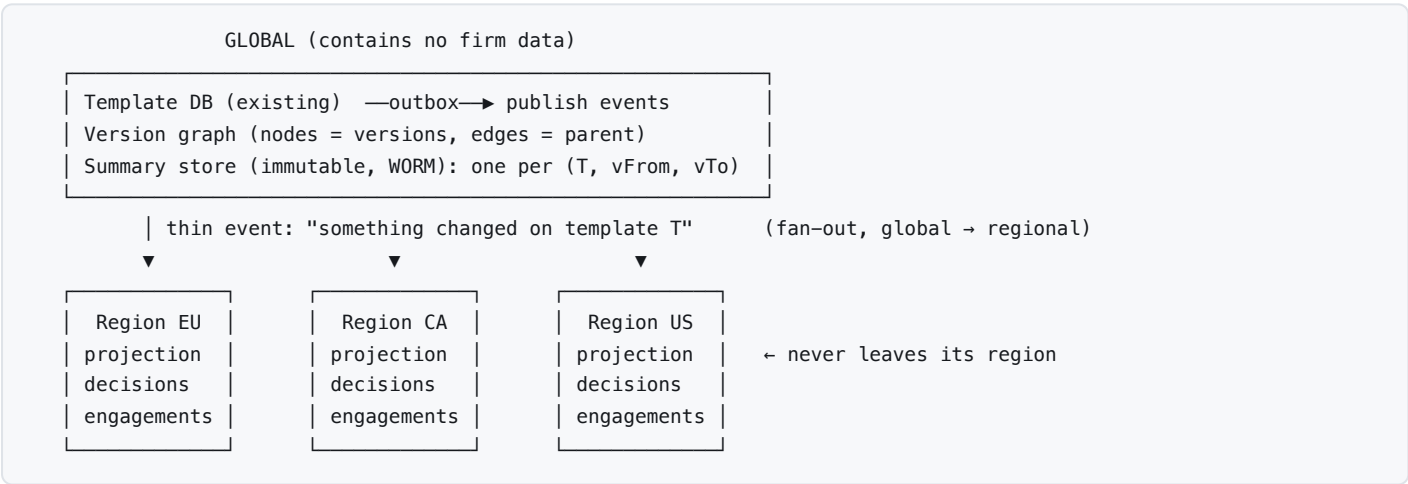
Reading an engagement's template version requires loading it: **~1 minute, hard constraint.**

Quantity	Arithmetic	Result
Load every engagement once	$800,000 \times 1 \text{ min}$	13,333 h $\approx$ 555 days of serial compute
Load per publish (20,000 eng/product)	$20,000 \times 1 \text{ min}$	333 h $\approx$ 14 days per publish
Load per publish, monthly	$40 \text{ products} \times 4.3 \text{ wk} \times 20,000 \times 1 \text{ min}$	~57,000 h/month $\approx$ 3.4M loads

Loading engagements on publish is $\sim 4\times$ the cost of rebuilding the entire estate every month. It can never catch up. **The 1-minute cost must be paid at most once per engagement, not once per publish.**

Everything below follows from that: we keep a small, queryable **projection** of the one field that lives inside an expensive-to-read object, and we never re-read the expensive object to answer a query.

1. Architecture and data ownership



Ownership rule: content that belongs to nobody travels; content that belongs to someone stays. Template content, diffs, publish events and summaries are identical for all firms (the template DB is already shared), so they are global. The projection and decision log are firm metadata — they reveal which engagements exist and what a firm chose — so they stay in-region. Only (template_id, version, parent_version, market, withdrawn) and summary text cross a border, and only global → regional: a property that is cheap to explain to an auditor.

Projection (per region) — current state, derived, rebuildable

Column	Source
engagement_id, firm_id	engagement lifecycle hook
template_id, applied_version, market_id	create / apply / roll-forward hooks, or a load
version_source (CONFIRMED / INFERRED / UNKNOWN), verified_at	how we learned it
is_archived	archive/close hook
pending_to_version, pending_summary_id, pending_detected_at	computed on publish

Read path. The engagement list is one indexed, paginated query on firm_id, returning each engagement's state plus a firm-level pending count — so the 40,000-engagement firm gets a page and a number, never a scan. Clicking one fetches the summary by pending_summary_id from the global store (shared by every firm on that version pair, so it caches). Neither view loads an engagement. Queries are firm-scoped from the caller's identity and the projection is partitioned by firm, so cross-firm reads are prevented by the data path, not by filtering afterwards.

Maintained by hooks on **engagement-management actions** — create, open/load, apply, decline, archive, delete, roll-forward — not by DB triggers: the stored form of an engagement is not a queryable record of its state, so only the action layer knows the version. Roll-forward matters: next year's engagement inherits the *source engagement's* version, not the latest published one; assuming "latest" corrupts the projection from day one.

Decisions (per region) — append-only facts, NOT rebuildable

decision_id, engagement_id, firm_id, template_id, from_version, to_version,
decision(APPLIED|DECLINED), summary_id, user_id, decided_at

The projection can be rebuilt by loading engagements; the decision log cannot be rebuilt by anything. **What can be rebuilt may be fragile; what cannot must be bulletproof** — decisions are written in the same transaction as the user action (outbox for the projection update), never via a best-effort hook.

summary_id is deliberately part of the record: in November, the defensible claim is not "the diff said X", it is "this user declined while reading *this* text".

Stack (AWS)

Concern	Service	Why
Projection	DynamoDB per region, PK firm_id / SK engagement_id, GSI template_id+market_id	The read path is firm-scoped and the fan-out scan template-scoped; both stay single-index queries
Decisions	Separate append-only DynamoDB table, PITR enabled	Different lifecycle: irreplaceable, so it is backed up rather than rebuilt
Publish events	Outbox in the template DB → EventBridge → one SQS queue per region, with DLQ	The outbox removes the dual-write; per-region queues keep fan-out inside the region
Fan-out + backfill workers	ECS Fargate consumers sharing one concurrency limiter	Downstream capacity is the scarce resource, so it is one knob, not per-service tuning
Summaries	S3 with Object Lock, metadata in DynamoDB, generation via Bedrock with a pinned model id	WORM storage is what makes November defensible

2. Version semantics: pending, declined, withdrawn

Versions form a **DAG**, not a line, so pending is an **ancestry** test, never a numeric comparison:

v1-v2-v3-v4-v5-v7
└─v6

(EU)
(CA)

Engagement on v6 (CA) has 6 < 7 but v7 is not a descendant of v6 → not pending. `version < 7` is wrong.

Pending(e, vNew) ⇒ template_id matches ∧ applied_version is an ancestor of vNew ∧ vNew not withdrawn ∧ not archived ∧ version_source = CONFIRMED . The diff shown is always applied_version → newest reachable non-withdrawn version , never between intermediate versions: templates are cumulative, so v5 containing v4 is one pending item, not two.

Declining pins a decision about a version, not a set of changes. If a firm declines v5 and v7 later arrives (descending from v5), v7 is offered, the summary is computed v3 → v7 (against what the engagement actually has), and it is labelled "*includes changes from v5, declined 12 Mar*". The rejected alternative — suppressing previously declined changes from the summary — produces a summary that lies: applying v7 delivers v5's content regardless, because templates are cumulative and partial application is out of scope. We chose the honest option and pay for it with user fatigue, mitigated by labelling.

Withdrawal is a flag, never a delete. The graph is append-only; deleting a node would orphan v7, break ancestry, and make March's summary unreproducible. On withdrawal: pending targets recompute to the newest non-withdrawn reachable version (possibly none); declined decisions stay as history; engagements that already applied it keep it and get a "based on a withdrawn version" flag.

Three-state indicator

UP TO DATE · UPDATE PENDING · VERIFYING (version not yet confirmed). The third state exists because the alternative is lying, and it doubles as the metric that tracks backfill completion.

3. Correctness and production evolution

Events give speed; reconciliation gives correctness. Both sides of the dual-write problem are covered without ordering guarantees:

- **Transactional outbox** on the template DB — the event is committed with the version, then published.
- **Thin events:** the event says only "template T changed"; the region re-reads authoritative state from the template DB. "v5 withdrawn" arriving before "v5 published" is a non-problem — the read returns current truth. No sequence numbers, no gap detection, no ordering window.
- **Idempotency:** key `(event_id, engagement_id)`; writes are state assignments (`pending_to_version = v7`), never increments. Duplicate delivery costs nothing — otherwise it would double-spend inference and the 1-minute downstream call.
- **Reconciliation (anti-entropy)** every few minutes: compare each region's known template heads against the template DB (40 products — effectively free) and recompute pending. This alone satisfies a minutes-level SLO, so a lost event self-heals and event delivery is an optimisation, not a correctness requirement.
- **Engagement-side truth:** the engagement is the source of truth for its version, and every open is a free audit — compare, heal on mismatch, emit the mismatch as the correctness SLI. `apply` is idempotent: re-applying an already-applied version heals the projection and does nothing else.

Asymmetry worth stating plainly: template-side reconciliation is free; engagement-side reconciliation costs 1 minute per row, so it can never run at full sweep. Correctness of `applied_version` therefore relies on hooks plus opportunistic verification on open — the weakest link in the design (§6).

Recovery. A lost regional projection is degraded, not wrong: flip rows to `VERIFYING` and re-run the backfill — users see "Verifying", never a false "up to date". The decision log has no rebuild path, which is why point-in-time recovery is a hard requirement there and nowhere else.

Migration and backfill

Infer, don't load, where possible: creation timestamp + publish history gives the version an engagement was created with, marked `INFERRED` and never shown as `UP TO DATE` — it is wrong exactly where the domain is interesting (roll-forward, branches, withdrawals), so it *prioritises* verification rather than answering the user. **Opportunistic confirmation** on every open costs nothing: the user already paid the minute. **Background backfill** uses the same capacity-limited downstream as the fan-out worker, ordered by recent activity rather than creation date, skipping archived engagements.

Arithmetic for the negotiation with the owning team: 10%/month of the estate = 80,000 loads/month = 80,000 min ÷ 43,200 min/month ≈ **2 concurrent loads sustained; ~20 concurrent finishes the whole estate in under a month.** The real question is not technical, it is how much of that team's capacity we are granted.

Rollout with no maintenance window: build the projection in **shadow mode** behind a per-firm feature flag and gate exposure on the correctness SLI. Enable internal/pilot → small → mid firms, the 40,000-engagement firm last: blast radius first, scale validated in shadow where nobody sees a wrong indicator. Rollback = flip the flag; the projection keeps building.

4. Scale, cost and operations

Item	Arithmetic	Monthly
Publishes	40 products × 4.3 wk	~172
Fan-out reads	projection scan per publish (indexed by <code>template_id, market</code>)	ms, not minutes
Summaries	≤ ~50 distinct source versions per publish (weekly cadence, ≤1-yr engagements); ~10 typical → 172 × 10	~1,720
Inference	50k in-tokens @ \$3/M + 1k out @ \$15/M ≈ \$0.165 each, ×2 for a validation pass	~\$570
Event bus	172 messages	< \$0.01
Projection storage	800k small rows	tens of dollars
Backfill (one-off)	13,333 h of downstream compute, ~20 concurrent for a month	one-off, dominated by the other team's capacity, not by us

Two things to notice. First, **the unit of summarisation is the version pair, not the engagement:** per-engagement summaries would be 172 publishes × 20,000 engagements × \$0.165 ≈ **\$568,000/month** — a **1,000×** difference, and it exists only because

template content is not firm-specific. Second, **cost is dominated by input tokens**, so the cheapest lever is diff size, not call count.

Against the \$X budget. Steady state is **~\$700/month all-in** — inference ~\$570, projection storage and event bus the rest — so inference is ~80% of the bill and the feature fits any budget from ~\$1k/month up. If \$X were tighter, the lever is granularity, not architecture: summarise only the adjacent jump and generate composite pairs on demand (~5x less inference). The one figure that could dwarf this is the 13,333 h backfill, but that is the owning team's existing capacity being scheduled rather than new spend — a negotiation, not a line item.

SLOs

SLI	SLO	Why this one
Freshness: template commit → pending visible on all affected engagements	99% < 5 min	Measured from commit , so a never-emitted event still burns the clock
Correctness: projection version vs actual version on engagement open	99.9% match	Every open is a free audit; false "up to date" is tracked separately as the severe class
Coverage: active engagements (opened < 90 d) in VERIFYING	< 1%	Tracks backfill against what users actually touch

Alarms: reconciliation lag, DLQ depth, downstream error rate and permit saturation, summary validation rejection rate, share of **INFERRED** rows.

5. Human-readable summaries

Code owns facts; the LLM owns prose. The JSON diff is parsed deterministically into change records (`change_id` , path, old, new); code renders every number, threshold, date and identifier; rules — not the model — assign severity (a materiality threshold change is HIGH regardless of how it reads). The LLM gets the structured change list and must cite `change_id` s in structured output.

Validation, all automatic, all pre-publication:

- **Omission** (the dangerous failure): every `change_id` in the diff must be cited; missing → reject.
- **Fabrication**: any cited id not in the diff, or any quoted value not matching the diff → reject.
- **Softening**: severity is code-assigned; the model cannot downgrade it.
- **Regression**: a golden set of human-reviewed diff→summary pairs gated on every prompt or model change.
- The technical diff is always one click away as the fallback.

A summary's id is deterministic — it is the (`template_id` , `from_version` , `to_version`) triple — so the fan-out records the pointer without waiting for generation, and the UI renders "Summary in preparation" until the object exists.

Human review is moved before publish, not into the request path. The content team already understands the change it is shipping, so the adjacent-jump summary (v6→v7) is generated and approved inside the publish workflow — summaries exist before the event fires and the 5-minute SLO is untouched. Composite pairs (v3→v7) are machine-generated, human-reviewed only at HIGH severity, and labelled when not. Indicator and summary are decoupled: the indicator never waits on a summary.

November defensibility. Summaries are never regenerated — regeneration is not reproducible (non-determinism, model deprecation, prompt drift). Each is stored immutably (S3 Object Lock) with exact text, input diff hash, from/to versions, model id, prompt version, timestamp, validation results and reviewer. Decisions reference `summary_id` . That chain answers "what did this user see, on what data, produced how, and why was it trusted". Retention follows workpaper retention (assumed 7 years): no summary or decision is deleted while an engagement still references it.

Assumptions

- Token prices are illustrative (\$3/\$1M in, \$15/\$1M out); the shape of the cost argument matters, not the figure. \$X is assumed ≥ ~\$1k/month.
- An active engagement spans ~1 year, bounding distinct source versions per publish to ~50.
- The 800k engagements spread roughly evenly across 40 products (~20k each); firm skew (40 → 40,000) is absorbed by paging, not by assumptions about distribution.
- Regions are EU, CA and a default region; adding one is configuration, not redesign.
- The owning team grants a fixed, negotiated share of load capacity (~20 concurrent).

- Hooks on engagement actions can be written transactionally (outbox), as the brief permits.
- Applying template content, and partial application of it, is out of scope, as stated.

6. Tradeoffs

Chosen	Rejected	Gain	Price
Precomputed projection	Load engagements on read	ms instead of 14 days per publish	Can go stale; reconciliation required
Thin events + reconciliation	Ordered, self-contained events	Immune to loss, duplication, reordering	Extra read of the template DB
Decline pins a version (honest)	Decline pins a change set	Summary never promises what apply can't deliver	Users re-see evaluated changes
Summary per version pair	Per engagement/firm	~\$570 vs ~\$568,000 per month	No firm-specific personalisation
Three-state indicator	Binary	Never asserts a falsehood	One more state to explain in the UI
Human review only for HIGH severity	Review everything	~80 reviews/day is not a job we can staff	Some summaries reach users unreviewed

The requirement I would challenge: "within seconds" — I would commit to **5 minutes**. Templates publish weekly, users take days to decide, and they do not know when a publish happened, so the difference is imperceptible. In exchange the system leans on reconciliation, which makes the indicator *correct* rather than merely fast. (Seconds is not expensive at 172 events/month; the cost is operational complexity and an on-call SLO measured in seconds.)

Riskiest to get wrong: a summary that silently omits a change. A slow indicator is visible; an omission is invisible — to the user in March and to us — until a regulator asks in November, and the user declined an update they were never told about. Hence: code-verified change coverage, code-assigned severity, immutable summary provenance, permanent access to the technical diff. Second place, and the part I trust least: engagement-side correctness of `applied_version`, because it is the one thing we cannot afford to reconcile at full sweep (§3).

Deliberately left out: partial accept/decline per change (users will ask first, but it collides with cumulative templates and with what apply can deliver — a product decision, not a patch); email/push channels (the in-list indicator covers the use case, and a weekly change does not justify one); multi-language summaries (market-specific content means this is coming, but not in v1); firm-specific personalisation (it would destroy the 1,000x cost reduction); automatic application (out of scope, and it removes the professional judgement the product exists to support).